Hearth — Database Design

> *Single source of truth for the PostgreSQL schema, data architecture decisions, and storage boundaries. This document covers the canonical `packages/db/prisma/schema.prisma` plus the rationale behind every significant design choice.*

---

1. What Lives Where

The database stores **metadata and structured records**. It does not store files.

```
┌──────────────────────────────────────────────────────────────────┐
│                        PostgreSQL (Neon → RDS)                   │
│                                                                  │
│  Users, CareRecipients, Memberships                              │
│  Documents (S3 key + metadata — never the file bytes)            │
│  Medications, Providers, Screenings                              │
│  Tasks, CareLogEntries                                           │
│  AI Conversations and Messages                                   │
│  Stage Assessments                                               │
│  AuditLogs (insert-only, 7-year HIPAA retention)                 │
│  ConsentRecords                                                  │
│  ShareLinks (token + expiry — Redis holds the TTL copy)          │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│                          Redis (Upstash → ElastiCache)           │
│                                                                  │
│  session:{userId}        → JWT payload cache (5 min TTL)         │
│  ratelimit:{userId}      → AI query counter (1 hr rolling window)│
│  share:{token}           → share link metadata + TTL (48 hr)     │
│  jwks:auth0              → Auth0 public keys (1 hr TTL)          │
│  suggest:{recipientId}   → AI suggested questions (24 hr TTL)    │
└──────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────┐
│                          AWS S3                                  │
│                                                                  │
│  documents/{careRecipientId}/{documentId}/{filename}             │
│  emergency-packets/{shareToken}.pdf                              │
└──────────────────────────────────────────────────────────────────┘
```

---

2. Schema Source of Truth

The canonical schema lives at `packages/db/prisma/schema.prisma`. It is reproduced in full here for reference; if the two ever diverge, the `.prisma` file wins.

```
generator client {
  provider = "prisma-client-js"
}
```

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ─── Enums ──────────────────────────────────────────────────────────────────

enum Role {
  ADMIN
  COORDINATOR
  VIEWER
}

enum DocumentCategory {
  LEGAL
  MEDICAL
  FINANCIAL
  INSURANCE
  IDENTITY
  CARE_PLANS
  OTHER
}

enum DocumentVisibility {
  ALL_MEMBERS
  ADMIN_ONLY
  SPECIFIC_MEMBERS  // requires DocumentPermissionMember rows
}

enum DocumentStatus {
  PENDING
  PROCESSING
  READY
  ERROR
}

enum MedicationFrequency {
  DAILY
  TWICE_DAILY
  THREE_TIMES_DAILY
  WEEKLY
  AS_NEEDED
  OTHER
}

enum CareStage {
  STAGE_1_INDEPENDENT
  STAGE_2_SOME_HELP
  STAGE_3_ASSISTED_LIVING
  STAGE_4_MEMORY_CARE
  STAGE_5_END_OF_LIFE
```

```
}

enum TaskStatus {
  TODO
  IN_PROGRESS
  DONE
}

enum TaskPriority {
  LOW
  MEDIUM
  HIGH
  URGENT
}

enum AuditAction {
  CREATE
  READ
  UPDATE
  DELETE
  DOWNLOAD
  SHARE
}

// ─── Core identity ──────────────────────────────────────────────────────

model User {
  id        String   @id @default(cuid())
  auth0Sub  String   @unique
  email     String   @unique
  name      String
  avatarUrl String?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  memberships         CareRecipientMembership[]
  uploadedDocuments   Document[]
  assignedTasks       Task[]                    @relation("AssignedTo")
  createdTasks        Task[]                    @relation("CreatedBy")
  careLogEntries      CareLogEntry[]
  auditLogs           AuditLog[]
  aiConversations     AiConversation[]
  consentRecords      ConsentRecord[]
  alerts              Alert[]
  agingPlan           AgingPlan?
  documentPermissions DocumentPermissionMember[]
}

model CareRecipient {
  id          String     @id @default(cuid())
  firstName   String
  lastName    String
```

```
  dateOfBirth DateTime?
  photoUrl    String?
  careStage   CareStage  @default(STAGE_1_INDEPENDENT)
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  deletedAt   DateTime?  // soft delete

  memberships      CareRecipientMembership[]
  documents        Document[]
  medications      Medication[]
  providers        Provider[]
  screenings       Screening[]
  tasks            Task[]
  careLogEntries   CareLogEntry[]
  aiConversations  AiConversation[]
  stageAssessments StageAssessment[]
  documentContacts DocumentContact[]
  alerts           Alert[]
  agingPlans       AgingPlan[]        // Phase 2: linked caregiver aging plans
}

model CareRecipientMembership {
  id              String        @id @default(cuid())
  userId          String
  careRecipientId String
  role            Role          @default(COORDINATOR)
  createdAt       DateTime      @default(now())

  user          User          @relation(fields: [userId], references: [id],
onDelete: Cascade)
  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id],
onDelete: Cascade)

  @@unique([userId, careRecipientId])
}

// ─── Documents ──────────────────────────────────────────────────────────

model Document {
  id              String           @id @default(cuid())
  careRecipientId String
  uploadedById    String
  filename        String
  s3Key           String           @unique
  mimeType        String
  sizeBytes       Int
  category        DocumentCategory   @default(OTHER)
  visibility      DocumentVisibility @default(ALL_MEMBERS)
  tags            String[]
  status          DocumentStatus     @default(PENDING)
  expiresAt       DateTime?
  notes           String?
```

```
  createdAt       DateTime           @default(now())
  updatedAt       DateTime           @updatedAt
  deletedAt       DateTime?          // soft delete

  careRecipient     CareRecipient            @relation(fields: [careRecipientId],
references: [id])
  uploadedBy        User                     @relation(fields: [uploadedById],
references: [id])
  shareLinks        ShareLink[]
  permissionMembers DocumentPermissionMember[]

  @@index([careRecipientId, category])
  @@index([expiresAt])
}

// Junction table for SPECIFIC_MEMBERS document visibility
model DocumentPermissionMember {
  id         String @id @default(cuid())
  documentId String
  userId     String

  document Document @relation(fields: [documentId], references: [id], onDelete:
Cascade)
  user     User     @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([documentId, userId])
}

// ShareLink supports multiple share types. resourceId is polymorphic:
// - EMERGENCY_PACKET: resourceId = careRecipientId
// - DOCUMENT: resourceId = documentId
// - CARE_SUMMARY: resourceId = careRecipientId
model ShareLink {
  id          String    @id @default(cuid())
  token       String    @unique @default(cuid())
  type        ShareType
  resourceId  String
  createdById    String
  expiresAt      DateTime
  pin            String?   // bcrypt-hashed PIN for Share Link mode
  viewLimit      Int?      // max access count (null = unlimited)
  accessCount    Int       @default(0)
  lastAccessedAt DateTime?
  revokedAt      DateTime?
  createdAt      DateTime  @default(now())

  // Optional FK to Document for DOCUMENT type links
  documentId  String?
  document    Document? @relation(fields: [documentId], references: [id])

  @@index([token])
  @@index([expiresAt])
```

```
}

// DocumentContact: non-user people who can be reminded to upload documents
// ("Remind Dorothy" — Dorothy is not a Hearth user)
model DocumentContact {
  id              String    @id @default(cuid())
  careRecipientId String
  name            String
  email           String?
  phone           String?
  relationship    String?   // "daughter", "attorney", etc.
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  careRecipient   CareRecipient       @relation(fields: [careRecipientId],
references: [id])
  reminders       DocumentReminder[]
}

// Tracks "Remind Dorothy" actions — who was reminded, for which doc, and when
model DocumentReminder {
  id          String    @id @default(cuid())
  contactId   String
  documentTag String    // e.g. "healthcare_proxy" — what doc was requested
  sentAt      DateTime  @default(now())
  respondedAt DateTime?
  channel     String    // "email" | "sms"

  contact DocumentContact @relation(fields: [contactId], references: [id])
}

// ─── Health Hub ──────────────────────────────────────────────────────────

model Medication {
  id              String              @id @default(cuid())
  careRecipientId String
  name            String
  dosage          String
  frequency       MedicationFrequency
  prescribedBy    String?
  startDate       DateTime?
  endDate         DateTime?
  refillDate      DateTime?
  notes           String?
  isActive        Boolean             @default(true)
  isBeersCriteria Boolean             @default(false)
  // FHIR-ready columns
  externalId      String?
  source          String?
  fhirJson        Json?
  lastSyncedAt    DateTime?
  createdAt       DateTime            @default(now())
```

```
  updatedAt       DateTime            @updatedAt
  deletedAt       DateTime?

  careRecipient         CareRecipient          @relation(fields: [careRecipientId],
references: [id])
  interactionsAsA       MedicationInteraction[] @relation("MedicationA")
  interactionsAsB       MedicationInteraction[] @relation("MedicationB")

  @@index([careRecipientId, isActive])
}

model MedicationInteraction {
  id            String            @id @default(cuid())
  medicationAId String
  medicationBId String
  severity      InteractionSeverity
  description   String
  source        String?           // e.g. "FDA", "DrFirst", "static_seed"
  createdAt     DateTime          @default(now())

  medicationA Medication @relation("MedicationA", fields: [medicationAId],
references: [id])
  medicationB Medication @relation("MedicationB", fields: [medicationBId],
references: [id])

  @@unique([medicationAId, medicationBId])
}

model Condition {
  id              String    @id @default(cuid())
  careRecipientId String
  name            String
  diagnosedAt     DateTime?
  notes           String?
  isActive        Boolean   @default(true)
  // FHIR-ready
  externalId      String?
  fhirJson        Json?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
}

model Allergy {
  id              String    @id @default(cuid())
  careRecipientId String
  allergen        String
  reaction        String?
  severity        InteractionSeverity?
  // FHIR-ready
  externalId      String?
```

```
  fhirJson        Json?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
}

model Provider {
  id              String    @id @default(cuid())
  careRecipientId String
  name            String
  specialty       String?
  phone           String?
  fax             String?
  address         String?
  organization    String?
  notes           String?
  isPrimary       Boolean   @default(false)
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime?

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
  appointments  Appointment[]
}

model Appointment {
  id              String    @id @default(cuid())
  careRecipientId String
  providerId      String?
  title           String
  scheduledAt     DateTime
  notes           String?
  prepChecklist   Json?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
  provider      Provider?     @relation(fields: [providerId], references: [id])

  @@index([careRecipientId, scheduledAt])
}

model Screening {
  id              String          @id @default(cuid())
  careRecipientId String
  name            String
  lastDate        DateTime?
  nextDueDate     DateTime?
  status          ScreeningStatus @default(UPCOMING)
  notes           String?
  // FHIR-ready
```

```
  externalId      String?
  fhirJson        Json?
  createdAt       DateTime        @default(now())
  updatedAt       DateTime        @updatedAt

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
}

// ─── Stage Navigator ──────────────────────────────────────────────────────

model StageAssessment {
  id              String    @id @default(cuid())
  careRecipientId String
  stage           CareStage
  assessedAt      DateTime  @default(now())
  notes           String?
  responses       Json?     // FHIR-ready: structured questionnaire responses

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])

  @@index([careRecipientId, assessedAt])
}

model WarningSignLog {
  id              String          @id @default(cuid())
  careRecipientId String
  loggedById      String
  domain          WarningSignDomain
  observation     String
  // urgency is system-derived from observation type — not user-entered
  observedAt      DateTime          @default(now())
  notes           String?
  resolvedAt      DateTime?

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])

  @@index([careRecipientId, domain])
}

// ─── My Aging Plan ────────────────────────────────────────────────────────

// AgingPlan belongs to a User (the caregiver planning for themselves).
// Phase 2: careRecipientId links to an actual CareRecipient record for the
caregiver.
// Nullable FK is intentional — data model must not preclude the Phase 2 two-record
pattern.
model AgingPlan {
  id              String    @id @default(cuid())
  userId          String    @unique  // one plan per user
  careRecipientId String?           // Phase 2: linked CareRecipient record for the
caregiver
  state           String?           // US state for advance directive templates
```

```
  completionPct   Int       @default(0)
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  user            User                  @relation(fields: [userId], references:
[id])
  careRecipient   CareRecipient?        @relation(fields: [careRecipientId],
references: [id])
  sections        AgingPlanSection[]
  journalEntries  WishesJournalEntry[]
  advDirective    AdvanceDirective?
}

model AgingPlanSection {
  id          String    @id @default(cuid())
  planId      String
  sectionKey  String    // "legal" | "care_preferences" | "financial" |
"conversation"
  completionPct Int     @default(0)
  data        Json?     // section-specific structured responses
  updatedAt   DateTime  @updatedAt

  plan AgingPlan @relation(fields: [planId], references: [id], onDelete: Cascade)

  @@unique([planId, sectionKey])
}

model AdvanceDirective {
  id          String    @id @default(cuid())
  planId      String    @unique
  state       String
  templateVersion String
  responses   Json      // wizard answers
  pdfS3Key    String?   // generated PDF stored in S3
  generatedAt DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  plan AgingPlan @relation(fields: [planId], references: [id])
}

model WishesJournalEntry {
  id        String    @id @default(cuid())
  planId    String
  prompt    String?   // optional journaling prompt used
  content   String
  isShared  Boolean   @default(false)
  createdAt DateTime  @default(now())
  updatedAt DateTime  @updatedAt

  plan AgingPlan @relation(fields: [planId], references: [id], onDelete: Cascade)
}
```

```
// ─── Family Workspace ──────────────────────────────────────────────────

model Task {
  id              String       @id @default(cuid())
  careRecipientId String
  createdById     String
  assignedToId    String?
  title           String
  description     String?
  status          TaskStatus   @default(TODO)
  priority        TaskPriority @default(MEDIUM)
  dueDate         DateTime?
  completedAt     DateTime?
  createdAt       DateTime     @default(now())
  updatedAt       DateTime     @updatedAt
  deletedAt       DateTime?

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
  createdBy     User          @relation("CreatedBy", fields: [createdById],
references: [id])
  assignedTo    User?         @relation("AssignedTo", fields: [assignedToId],
references: [id])

  @@index([careRecipientId, status])
  @@index([assignedToId, status])
}

model CareLogEntry {
  id              String           @id @default(cuid())
  careRecipientId String
  authorId        String
  content         String
  category        CareLogCategory?
  createdAt       DateTime         @default(now())
  updatedAt       DateTime         @updatedAt

  careRecipient CareRecipient @relation(fields: [careRecipientId], references: [id])
  author        User          @relation(fields: [authorId], references: [id])

  @@index([careRecipientId, createdAt])
  @@index([careRecipientId, category])
}

// ─── Alerts ────────────────────────────────────────────────────────────

model Alert {
  id              String       @id @default(cuid())
  careRecipientId String?      // null for user-level alerts
  userId          String       // which user sees this alert
  type            AlertType
  priority        AlertPriority
```

```
  title           String
  body            String?
  actionUrl       String?      // deep link to the relevant resource
  resourceType    String?      // "Document" | "Medication" | "Screening" etc.
  resourceId      String?
  isRead          Boolean      @default(false)
  readAt          DateTime?
  isDismissed     Boolean      @default(false)
  dismissedAt     DateTime?
  snoozedUntil    DateTime?
  expiresAt       DateTime?    // alert auto-expires if underlying issue resolved
  createdAt       DateTime     @default(now())

  careRecipient CareRecipient? @relation(fields: [careRecipientId], references:
[id])
  user          User           @relation(fields: [userId], references: [id])

  @@index([userId, isDismissed, isRead])
  @@index([careRecipientId, isDismissed])
  @@index([expiresAt])
}

// ─── AI Assistant ──────────────────────────────────────────────────────────

model AiConversation {
  id              String    @id @default(cuid())
  careRecipientId String?   // null for general (non-scoped) conversations
  userId          String
  title           String?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  careRecipient CareRecipient? @relation(fields: [careRecipientId], references:
[id])
  user          User           @relation(fields: [userId], references: [id])
  messages      AiMessage[]
}

model AiMessage {
  id             String   @id @default(cuid())
  conversationId String
  role           String   // "user" | "assistant" — string not enum for API
compatibility
  content        String   // stores stripped content only — never raw user input
with PHI
  createdAt      DateTime @default(now())

  conversation AiConversation @relation(fields: [conversationId], references: [id],
onDelete: Cascade)
}

// ─── Compliance ────────────────────────────────────────────────────────────
```

```
// Insert-only. Never update or delete rows. Retained 7 years (HIPAA).
model AuditLog {
  id           String      @id @default(cuid())
  userId       String?
  action       AuditAction
  resourceType String
  resourceId   String
  ipAddress    String?
  userAgent    String?
  metadata     Json?
  createdAt    DateTime    @default(now())

  @@index([userId])
  @@index([resourceType, resourceId])
  @@index([createdAt])
}

// Immutable legal record of user consent. Never update or delete.
model ConsentRecord {
  id          String   @id @default(cuid())
  userId      String
  consentType String   // "terms_of_service" | "hipaa_notice" | "marketing"
  version     String
  acceptedAt  DateTime @default(now())
  ipAddress   String?

  user User @relation(fields: [userId], references: [id])

  @@unique([userId, consentType, version])
  @@index([userId])
}

// ─── Enums (additional) ──────────────────────────────────────────────────────

enum ShareType {
  EMERGENCY_PACKET
  DOCUMENT
  CARE_SUMMARY
}

enum InteractionSeverity {
  LOW
  MODERATE
  HIGH
}

enum ScreeningStatus {
  OVERDUE
  DUE_THIS_YEAR
  UPCOMING
  COMPLETED
```

```
}

enum WarningSignDomain {
  COGNITIVE
  PHYSICAL
  FUNCTIONAL
  FINANCIAL
  SOCIAL
}

enum WarningUrgency {
  MONITOR
  SEE_DOCTOR
  ACT_NOW
}

enum CareLogCategory {
  COGNITIVE
  MEDICAL
  PHYSICAL
  NUTRITIONAL
  SOCIAL
  FINANCIAL
  BEHAVIORAL
  LEGAL_ADMINISTRATIVE
  OTHER
}

enum AlertType {
  DOCUMENT_MISSING
  DOCUMENT_EXPIRING
  DOCUMENT_NEEDS_SIGNATURE
  MEDICATION_LOW_REFILL
  SCREENING_OVERDUE
  DRUG_INTERACTION
  TASK_OVERDUE
  COVERAGE_EXPIRING
  MEDICARE_OEP
  STAGE_CHANGE_SUGGESTED
  SHARE_LINK_ACCESSED
  COLLABORATOR_JOINED
  DOCUMENT_UPLOADED_BY_OTHER
  FAMILY_MEETING_RECOMMENDED
}

enum AlertPriority {
  INFO       // FYI — no action required
  LOW        // should address within a week
  MEDIUM     // should address within a few days
  HIGH       // should address today
```

```
  URGENT     // act now
}
```

---

3. Design Decisions and Rationale

3a. IDs: CUID over UUID

All primary keys use `@default(cuid())`. CUIDs are URL-safe (no hyphens that need encoding), monotonically increasing within a process (better B-tree index locality than random UUIDs), and collision-resistant across distributed systems. Switch to UUIDs only if interoperability with an external system requires them.

3b. Soft Deletes

Models that represent user-created data carry a `deletedAt DateTime?` field:

Model	Soft delete	Why
`CareRecipient`	Yes	Regulatory and audit trail
`Document`	Yes	HIPAA — records must be recoverable
`Medication`	Yes	Clinical history should not be destroyed
`Provider`	Yes	History and audit
`Task`	Yes	Workspace history
`AuditLog`	No	Insert-only by design; never deleted
`ConsentRecord`	No	Legal record; never deleted
`ShareLink`	No	Expiry handled by `expiresAt`
`AiMessage`	No	Cascade-deletes with conversation

Prisma middleware appends `WHERE deletedAt IS NULL` to all `findMany`, `findFirst`, and `findUnique` calls on soft-deletable models.

3c. Role-Based Access: Membership Table

Access control is modeled through `CareRecipientMembership` rather than a single role column on `User`. A user can be `ADMIN` on their mother's record and `VIEWER` on their father's record independently.

Role	Capabilities
`ADMIN`	Full access: add/edit/delete any records, invite members, create share links, manage finances
`COORDINATOR`	Read/write: upload documents, add medications, create tasks, write care log entries, delete their own uploads
`VIEWER`	Read-only: cannot upload, edit, or share

Every Route Handler must verify the requesting user has a `CareRecipientMembership` row for the target `careRecipientId`. Queries not scoped to the user's memberships are a security defect.

3d. Documents: Metadata Only

The `Document` table stores metadata. File bytes are never written to the database. The `s3Key` is the pointer to the S3 object. The upload Route Handler generates a presigned URL; the browser uploads directly to S3. This keeps rows small, eliminates function memory limits as a constraint, and makes S3 the single source of truth for file data.

3e. ShareLink: Polymorphic Resource + Dual Storage

`ShareLink` uses a polymorphic `resourceId` rather than a hard `documentId` foreign key. This supports all three `ShareType` values:

- `EMERGENCY_PACKET` → `resourceId` = `careRecipientId`
- `DOCUMENT` → `resourceId` = `documentId` (optional `documentId` FK for referential integrity)
- `CARE_SUMMARY` → `resourceId` = `careRecipientId`

Dual storage (DB + Redis): PostgreSQL holds the persistent record; Redis holds a copy with a TTL matching `expiresAt`. On resolution, check Redis first — if the key is missing, return 410 Gone without a DB query. This pattern avoids full-table scans for expired links and removes the need for a cleanup cron job.

3f. DocumentContact: The "Dorothy" Problem

"Dorothy" is a non-user who the system can remind to upload a document. She does not have a Hearth account. `DocumentContact` stores her name, email, and relationship to the care recipient. `DocumentReminder` tracks each "remind" action sent (channel, when, whether she responded). This allows the caregiver to see "reminder sent 3 days ago, no response" without requiring Dorothy to create an account.

3g. AgingPlan: Caregiver as Their Own Care Recipient

A caregiver's personal aging plan (`AgingPlan`) belongs to a `User`, not a `CareRecipient`. For beta, `AgingPlan` is a parallel structure that mirrors the care-recipient concept without requiring the caregiver to "care for themselves" through the same membership/permission model.

Phase 2 introduces a two-record pattern: the caregiver creates a `CareRecipient` record for themselves and links it via the nullable `careRecipientId` FK on `AgingPlan`. The field is nullable from the start so no future migration is needed to add it. When `careRecipientId` is null, the plan is beta-mode (caregiver only); when set, the plan is linked to a full care recipient record.

Permission note: a user's `AgingPlan` is fully private by default. Sharing individual `WishesJournalEntry` rows is opt-in via `isShared`.

3h. AuditLog: Insert-Only

`AuditLog` is append-only. No `UPDATE` or `DELETE` operations are ever issued against it. Retention: 7 years (HIPAA). Every PHI access writes an audit row: document view/download/share, medication changes, AI query execution (action only — never log query content, which may contain PHI).

3i. Alert: Per-User, Not Per-Care-Recipient

Alerts belong to a `userId`, not just a `careRecipientId`. This is intentional: different family members should see different alert states. If ADMIN dismisses an alert about a missing document, that dismissal

should not propagate to other members — they may want to see it. Each member manages their own alert inbox. The `careRecipientId` is still indexed for "show all alerts for Ruth" filtering.

3j. AiMessage.content: Stripped Content Only

`AiMessage.content` stores the post-PII-stripping version of user messages. The raw user query (which may contain PHI) is never written to the database. The stripping pipeline in `src/lib/ai/pii.ts` runs before both the Claude API call and the database write. This is a hard invariant that must be enforced at the application layer.

3k. CareStage: Enum Over Integer

`CareStage` uses a PostgreSQL enum rather than integers 1–5. The database rejects invalid values, migration diffs are human-readable, and the labels self-document the domain model. Default is `STAGE_1_INDEPENDENT` — the safest default when no assessment has been completed.

3l. FHIR-Ready Columns

Health-related tables (`Medication` , `Condition` , `Allergy` , `Screening`) carry `externalId` , `source` , `fhirJson` , and `lastSyncedAt` columns. These cost nothing now and prevent schema migrations when FHIR import ships in Phase 2. The `fhirJson` column stores the raw FHIR resource JSON for reference; structured data still lives in the typed columns.

---

4. Indexes

Explicit indexes are defined where query patterns require them:

Table	Index	Query pattern
`Document`	`(careRecipientId, category)`	Filter vault by category
`Document`	`expiresAt`	Find expiring documents for cron job
`AuditLog`	`userId`	Show all actions by this user
`AuditLog`	`(resourceType, resourceId)`	Show all access to document X
`AuditLog`	`createdAt`	Date-range queries for HIPAA reports
`Task`	`(careRecipientId, status)`	Open tasks per care recipient
`Task`	`(assignedToId, status)`	My open tasks
`CareLogEntry`	`(careRecipientId, createdAt)`	Recent log entries per recipient
`CareLogEntry`	`(careRecipientId, category)`	Filter log by category
`Alert`	`(userId, isDismissed, isRead)`	My unread alert inbox
`Alert`	`(careRecipientId, isDismissed)`	All active alerts for Ruth
`ShareLink`	`token`	Share link resolution (hot path)
`ShareLink`	`expiresAt`	Cleanup expired links

`Appointment`	`(careRecipientId, scheduledAt)`	Upcoming appointments
`Medication`	`(careRecipientId, isActive)`	Active medication list
`StageAssessment`	`(careRecipientId, assessedAt)`	Stage history

Add indexes only when slow queries are observed. Do not add preemptively.

---

5. Encryption

At Rest

PostgreSQL data at rest is encrypted by Neon (Stage 1) and RDS (Stage 2+) at the storage layer. Transparent to the application.

In Transit

All connections use TLS. `DATABASE_URL` includes SSL parameters. Prisma enforces this.

Application-Layer Encryption (Stage 2+)

Highly sensitive PII fields will be encrypted at the application layer using AES-256-GCM + AWS KMS envelope encryption before being written to the database. Implemented in `packages/db/src/crypto.ts`.

Fields targeted for application-layer encryption when added:

- Any `ssnLast4` or `taxId` on `CareRecipient`
- Any `accountNumber` fields in Money & Coverage
- `AdvanceDirective.responses` (contains personal wishes data)

---

6. What's Not In the Schema Yet

These tables depend on unresolved product decisions or are intentionally deferred to Phase 2+:

Domain	Missing table	Decision needed
Health Hub	`DrugInteractionRule`	Which clinical data source? (static FDA seed vs. DrFirst API)
Health Hub	`BeersListEntry`	Same data source question
Care Finder	`SavedFacility`, `FacilityShortlist`	Which facility database? (Google Places vs. licensed eldercare DB)
Money & Coverage	`InsuranceCoverage`, `CostEstimatorInput`	What financial data to store? (privacy + liability review needed)
Money & Coverage	`BenefitItem`	Benefits database source? (Benefits.gov API vs. curated)
Messaging	`InboxMessage`, `MessageThread`	Phase 3 feature — full in-app messaging deferred

i18n	`LocalePreference`	Deferred — next-intl handles locale at runtime; no DB row needed for beta
Activity	`ActivityFeedEvent`	Phase 2 — activity feed deferred

---

7. Migration Path from Current Schema

The current `packages/db/prisma/schema.prisma` (as of April 2026) is missing relative to this full design:

New tables to add (all migrations run upfront in Phase 0 — see IMPLEMENTATION_PLAN.md):

- `DocumentPermissionMember` (SPECIFIC_MEMBERS visibility junction)
- `DocumentContact` + `DocumentReminder` (the "Dorothy" reminder flow)
- `MedicationInteraction` (drug interaction flagging)
- `Condition` + `Allergy` (Health Hub; emergency packet)
- `Appointment` (Health Hub; feeds NEEDS ATTENTION alerts)
- `WarningSignLog` (Stage Navigator)
- `AgingPlan` + `AgingPlanSection` + `AdvanceDirective` + `WishesJournalEntry` (My Aging Plan)
- `Alert` (unified alerts module)
- `StageAssessment` history table

Columns to add to existing tables:

- `Document.visibility` (DocumentVisibility enum — replaces `aiCategorized` which is removed)
- `Document.tags` (if not present)
- `Medication.isBeersCriteria` + FHIR columns
- `ShareLink.pin` + `ShareLink.viewLimit` + `ShareLink.revokedAt`
- `ShareLink` — change from `documentId` FK to polymorphic `(type, resourceId)` model
- `CareLogEntry.category` (enum)
- `AgingPlan.careRecipientId` (nullable FK — Phase 2 pre-condition)

Enum changes:

- `Role`: `MEMBER` → `COORDINATOR`
- `DocumentCategory`: add `CARE_PLANS`
- `DocumentVisibility`: new enum (`ALL_MEMBERS`, `ADMIN_ONLY`, `SPECIFIC_MEMBERS`)
- `ShareType`: needs `EMERGENCY_PACKET` and `CARE_SUMMARY` values
- `AlertType` and `AlertPriority`: entirely new
- `WarningSignDomain` (was `WarningSIgnDomain` — fix typo): `NUTRITIONAL` → `FUNCTIONAL`
- `WarningUrgency`: new (system-assigned urgency levels — not user-entered)
- `CareLogCategory`: add `BEHAVIORAL`, `LEGAL_ADMINISTRATIVE`

Migration order (all 9 run in Phase 0 before Sprint 1):

1. Core schema — users, care recipients, memberships, base enums
2. Documents — Document, ShareLink polymorphic model, DocumentPermissionMember, DocumentContact, DocumentReminder
3. Alerts — Alert, AlertType, AlertPriority enums
4. Tasks + CareLog — Task, CareLogEntry with updated CareLogCategory

5. Health Hub — Medication (with FHIR cols), Condition, Allergy, Provider, Appointment, Screening, MedicationInteraction
6. Stage Navigator — StageAssessment, WarningSignLog, WarningSignDomain, WarningUrgency
7. My Aging Plan — AgingPlan (with careRecipientId FK), AgingPlanSection, AdvanceDirective, WishesJournalEntry
8. AI Assistant — AiConversation, AiMessage
9. Compliance — AuditLog, ConsentRecord

---

8. Migration Strategy

```
# Create a new migration
pnpm --filter @hearth/db db:migrate:dev --name descriptive-name

# Apply pending migrations (CI + production deploy)
pnpm --filter @hearth/db db:migrate:deploy
```

Rules:

1. Never edit a committed migration file. Write a new corrective migration.
2. Additive changes (new table, nullable column) are safe to deploy without downtime.
3. Breaking changes (rename/drop column) require a two-phase deploy: add new → backfill → drop old.
4. Soft delete columns are permanent once added.

Environment promotion:

Environment	When migrations run
Local	Manually via `db:migrate:dev`
Preview (PR)	GitHub Actions against Neon preview branch
Staging	GitHub Actions on push to `staging`
Production	Netlify deploy hook before app starts

---

9. HIPAA Considerations

Safeguard	Implementation
Access control	`CareRecipientMembership` scopes every query
Audit controls	`AuditLog` insert-only; 7-year retention
Integrity	TLS in transit; storage encryption at rest
Minimum necessary	Never `SELECT *` in application code
Data backup	Neon automated backups (Stage 1); RDS snapshots + PITR (Stage 2)

Required before real PHI (not yet implemented):

- Application-layer encryption for high-sensitivity fields
- AWS BAA signed
- Formal access termination procedure for removed family members
- `AiMessage.content` stripping verified in CI (automated test)